

- Propuesta Técnica: API REST para Catálogos SAT
 - 1. Propuesta
 - 2. Análisis de la Situación Actual
 - 2.1 Script Existente
 - 2.2 Objetivos de la Migración
 - 3. Arquitectura de la API
 - 3.1 Diseño de Endpoints
 - Grupo 1: Gestión de Catálogos (/api/catalog)
 - Grupo 2: Consulta por Entidad (/api/{entity})
 - 3.2 Arquitectura de Seguridad
 - 3.3 Arquitectura de Datos
 - MongoDB - Estructura General
 - Optimización de Consultas
 - 4. Implementación Técnica
 - 4.1 Stack Tecnológico
 - Framework Web
 - Base de Datos
 - Procesamiento de Archivos
 - Autenticación y Seguridad
 - 4.2 Estructura del Proyecto
 - 4.3 Migración de Lógica Existente
 - Aprovechamiento del Script Actual
 - Nuevas Funcionalidades
 - 5. Flujos de Trabajo Detallados
 - 5.1 Proceso de Carga de Catálogos
 - 5.2 Consulta de Entidades
 - 5.3 Búsqueda Avanzada

Propuesta Técnica: API REST para Catálogos SAT

Versión: 1.0 **Fecha:** Julio 2025 **Proyecto:** Migración de Script Python a API REST
Tecnologías: FastAPI + MongoDB + Motor

1. Propuesta

Esta propuesta define la migración del script Python existente (converter.py) hacia una API REST completa que permita:

- **Carga automatizada** de archivos Excel SAT via endpoint `/load`
- **Almacenamiento estructurado** en MongoDB
- **Endpoints dinámicos** por entidad para consulta y búsqueda
- **Autenticación JWT** integrada con nd-auth-core

Alcance del proyecto:

- Migración completa del script existente
- API REST con autenticación
- Documentación interactiva
- Sistema listo para producción

2. Análisis de la Situación Actual

2.1 Script Existente

Archivos actuales:

- `converter.py` - Lógica principal de conversión Excel a JSON
- `usoconverter.py` - Script de procesamiento masivo
- Funcionalidad probada y estable

2.2 Objetivos de la Migración

Automatización completa:

- Upload de archivos via API
- Procesamiento automático
- Respuesta inmediata con resultados

Acceso programático:

- Endpoints REST para cada catálogo
- Búsqueda avanzada por criterios
- Integración con otros sistemas

Persistencia y gestión:

- Almacenamiento en MongoDB
 - Versionado de catálogos
 - Auditoría de cambios
-

3. Arquitectura de la API

3.1 Diseño de Endpoints

Grupo 1: Gestión de Catálogos (**/api/catalog**)

POST **/api/catalog/load**

- **Propósito:** Cargar archivo Excel y procesarlo
- **Input:** Archivo Excel (.xlsx/.xls) via multipart/form-data
- **Proceso:**
 1. Validación de formato y versión Excel
 2. Conversión automática si es necesario (.xls → .xlsx)
 3. Extracción de hojas con "&" en el nombre
 4. Procesamiento usando lógica de converter.py
 5. Almacenamiento en MongoDB
- **Output:** Lista de catálogos creados con estadísticas

GET **/api/catalog**

- **Propósito:** Listar todos los catálogos disponibles
- **Output:** Metadatos de catálogos (nombre, registros, fecha carga)

Grupo 2: Consulta por Entidad (**/api/{entity}**)

GET **/api/{entity}**

- **Propósito:** Obtener todas las entradas de un catálogo

- **Parámetros:** limit, offset para paginación
- **Ejemplo:** `/api/c_FormaPago?limit=50&offset=0`

GET `/api/{entity}/{id}`

- **Propósito:** Obtener entrada específica por código
- **Ejemplo:** `/api/c_FormaPago/01` (donde "01" es el código SAT)

GET `/api/{entity}/search`

- **Propósito:** Búsqueda avanzada dentro del catálogo
- **Parámetros:** q (query), field (campo específico), exact (búsqueda exacta)
- **Ejemplo:** `/api/c_FormaPago/search?q=efectivo&field=description`

3.2 Arquitectura de Seguridad

Componentes:

- **Middleware JWT** - Validación de tokens en todas las requests
- **nd-auth-core Integration** - Servicio externo de autenticación
- **Rate Limiting** - Protección contra abuso

Flujo de autenticación:

1. Cliente obtiene token JWT de nd-auth-core
2. Incluye token en header: `Authorization: Bearer <token>`
3. Middleware valida token con nd-auth-core
4. Si válido (200 OK), permite acceso a endpoints
5. Si inválido, retorna 401 Unauthorized

3.3 Arquitectura de Datos

MongoDB - Estructura General

Base de Datos: `catalogs_sat`

Colección principal: `catalogs`

- Almacena los catálogos SAT completos
- Un documento por catálogo procesado

- Estructura flexible para diferentes tipos de catálogos

Elementos principales de cada documento:

- Nombre del catálogo (ej: "c_FormaPago")
- Archivo fuente de origen
- Lista de entradas con sus campos
- Metadatos de procesamiento
- Timestamps de creación y actualización

Optimización de Consultas

- Índices por nombre de catálogo para acceso rápido
 - Índices compuestos para búsquedas por código
 - Índices de texto para búsqueda full-text en descripciones
 - Índices temporales para ordenamiento por fecha
-

4. Implementación Técnica

4.1 Stack Tecnológico

Framework Web

- **FastAPI 0.104+** - Framework moderno con validación automática
- **Uvicorn** - Servidor ASGI de alto rendimiento
- **Pydantic** - Validación de datos y schemas

Base de Datos

- **MongoDB 7.0+** - Base de datos NoSQL para flexibilidad
- **Motor 3.3+** - Driver asíncrono para MongoDB
- **PyMongo 4.5+** - Driver base para operaciones síncronas

Procesamiento de Archivos

- **Pandas 2.1+** - Reutilización de lógica existente
- **OpenPyXL 3.1+** - Lectura de archivos Excel

- **XlsxWriter** - Conversión de formatos si es necesario

Autenticación y Seguridad

- **python-jose** - Manejo de tokens JWT
- **httpx** - Cliente HTTP para nd-auth-core
- **python-multipart** - Upload de archivos

4.2 Estructura del Proyecto

```

ConversorCatalogos/
├── app/
│   ├── main.py                # Punto de entrada FastAPI
│   ├── core/
│   │   ├── config.py          # Configuración general
│   │   ├── security.py         # JWT y autenticación
│   │   └── database.py         # Conexión MongoDB
│   ├── api/
│   │   ├── v1/
│   │   │   ├── endpoints/
│   │   │   │   ├── catalog.py    # Endpoints /api/catalog/*
│   │   │   │   └── entities.py   # Endpoints /api/{entity}/*
│   │   │   └── dependencies.py    # Dependency injection
│   │   └── middleware/
│   │       ├── auth.py           # Middleware JWT
│   │       └── cors.py           # CORS configuration
│   ├── models/
│   │   ├── catalog.py           # Modelos Pydantic
│   │   └── responses.py         # Schemas de respuesta
│   ├── services/
│   │   ├── catalog_service.py    # Lógica de negocio
│   │   ├── excel_processor.py    # Migración de converter.py
│   │   └── auth_service.py       # Integración nd-auth-core
│   └── utils/
│       ├── file_utils.py        # Utilidades de archivos
│       └── validators.py        # Validaciones personalizadas
├── tests/
│   ├── test_catalog_endpoints.py
│   ├── test_entity_endpoints.py
│   └── test_excel_processor.py
├── legacy/
│   ├── converter.py             # Script original (referencia)
│   └── usoconverter.py          # Script masivo (referencia)
├── requirements.txt
├── .env.example
└── README.md

```

4.3 Migración de Lógica Existente

Aprovechamiento del Script Actual

- **converter.py** → **excel_processor.py**
 - Mantener funciones core: `leer_excel()`, `limpiar_dataframe()`, `convertir_a_json()`
 - Adaptar para uso asíncrono
 - Agregar validaciones adicionales
- **usoconverter.py** → **catalog_service.py**
 - Lógica de procesamiento masivo
 - Manejo de múltiples hojas
 - Generación de reportes

Nuevas Funcionalidades

- **Validación de versiones Excel** - Detección automática .xls vs .xlsx
 - **Conversión automática** - Upgrade de formatos antiguos
 - **Persistencia MongoDB** - Reemplazo de archivos JSON
 - **APIs RESTful** - Acceso programático a datos
-

5. Flujos de Trabajo Detallados

5.1 Proceso de Carga de Catálogos

Flujo del endpoint **POST** `/api/catalog/load`:

1. **Validación JWT** - Middleware verifica token con nd-auth-core
2. **Validación de archivo** - Verificar formato (.xlsx/.xls) y tamaño
3. **Conversión automática** - Si es .xls, convertir a .xlsx
4. **Detección de hojas** - Identificar hojas con "&" en el nombre
5. **Procesamiento por hoja** - Extraer datos, limpiar y validar
6. **Persistencia** - Guardar en colección MongoDB
7. **Respuesta** - Retornar resumen de operación con estadísticas

5.2 Consulta de Entidades

Flujo del endpoint **GET /api/{entity}**:

Flujo interno:

1. **Validación JWT** - Verificar autenticación
2. **Validación de entidad** - Verificar que el catálogo existe
3. **Query MongoDB** - Buscar en colección por catalog_name
4. **Paginación** - Aplicar limit/offset a entries array
5. **Formato de respuesta** - Estructurar datos para cliente

Flujo del endpoint **GET /api/{entity}/{id}**:

1. **Validación JWT** - Verificar autenticación
2. **Query específico** - Buscar entrada por código exacto
3. **Validación de existencia** - Retornar 404 si no existe
4. **Respuesta** - Retornar datos de la entrada

5.3 Búsqueda Avanzada

Flujo del endpoint **GET /api/{entity}/search**:

1. **Validación JWT** - Verificar autenticación
2. **Construcción de query** - Crear aggregation pipeline MongoDB
3. **Tipos de búsqueda:**
 - **Campo específico:** Buscar solo en field indicado
 - **Búsqueda global:** Buscar en todos los campos de texto
 - **Exacta vs fuzzy:** Usar regex o match exacto
4. **Ejecución** - Correr agregación en MongoDB
5. **Formato de respuesta** - Retornar resultados ordenados por relevancia